

Functions

ID1063 Introduction to Programming in C

Instructor: Rakesh Venkat

Week 7 • Lectures 13,14

Week 7 at a glance

Lecture 13: functions and control flow

Lecture 14: functions with arrays and strings

Written midsem: 16 Sep

Lab Exam 1: 19 Sep

No Week 8 lab

Please fill out the form shared by Prof. Aravind yesterday regarding how you are finding the course until now.

A simple problem

- Print a clean log report with a row of stars between each line

```
*****  
  
    ATMOSPHERIC DATA LOG  
  
*****  
  
Status: All sensors operational.  
  
*****  
  
    END OF REPORT  
  
*****
```

- How would you print every row of *'s?

Functions

- Functions are named, reusable blocks of code that accomplish a certain task

Instead of one long main:

- name a task, write function for it
- test it in isolation
- reuse it when needed

Familiar functions

```
printf("Hello\n");  
scanf("%d", &age);
```

- Code for these functions have already been written in the stdio library

main() : is a special function, and used by the Operating System when executing the program

How to write a function

return type	name	parameters/arguments
↓	↓	↓
<pre>int square(int value) { return value * value; }</pre>		

- return type: what the function gives as a result
- name: name of the function
- parameters or arguments: What the function acts on
- Body: the statements that do the job.

Example 1: First use of functions

```
#include <stdio.h>

void showWelcome(void) {
    printf("Welcome to ID1063!\n");
}

int main(void) {
    showWelcome();
    return 0;
}
```

void : means no value is used/returned

Function execution

1. main reaches call to showWelcome()
2. control moves into showWelcome
3. showWelcome function body is executed
4. control returns to main, proceeds execution from next line

```
#include <stdio.h>

void showWelcome(void) {
    printf("Welcome to ID1063!\n");
}

int main(void) {
    showWelcome();
    return 0;
}
```


Why use functions

- **Avoid repetition:** Functions allow you to write code once and use it many times.
- **Readability:** `main()` now reads like plain English (`printBorder()`), making the core program clear
- **Maintainability:** To change the border style or length, you only modify the code inside `printBorder()` once.

Example 2: Functions with return values

```
int square(int value) {  
    return value * value;  
}
```

```
int main(void) {  
    //take input from user  
    //call the function here to compute n^2  
}
```

Functions may compute a value that the main program can use

These are stored in a predetermined location that main knows about.

Function execution

1. main reaches call to square(...)
2. control moves into square
3. square is computed and is stored in a predetermined location known to main (return statement)
4. control returns to main, proceeds execution from next line

return is not printf

return value * value;

- sends a result back to the caller

printf("%d\n", value);

- displays characters on screen

A printed result is not automatically stored.

Definition vs call

Definition: describes the job once

```
int square(int value) {  
    return value * value;  
}
```

Call: uses the job

```
int answer = square(7);
```

Example 3: Finding the maximum

```
int maximum(int first, int second) {  
    if (first > second) {  
        return first;  
    }  
    return second;  
}  
  
//in main:  
int largest = maximum(73, 68);
```

Parameters vs arguments

Definition:

```
int maximum(int first, int second)
```

Call:

```
maximum(scoreA, scoreB)
```

parameter = variable in the definition

argument = value or expression in the call

Local variables

```
int square(int value) {  
    int ans = value * value;  
    return ans;  
}
```

Q: Is the variable 'ans' available in the main program? If we did `printf("%d", ans)` in the main program, what would happen?

Pass by value

```
void addOne(int number) {  
    number = number + 1;  
}  
  
int main(void) {  
    int count = 4;  
    addOne(count);  
    printf("Count value is: %d \n",count);  
    return 0;  
}
```

Q: What is printed by the program?

A: Normal variables are passed by *value*: i.e. **only a copy is passed to the function**

So, the printed output is still 4, since addOne works on the copy of the count variable.